
WANKRBOT

Multi-LLM Grading Pipeline

Annotation Rotation Mechanism v2

FIELD	VALUE
Document Type	Training Pipeline Specification
Version	2.0
Date	February 22, 2026
Classification	Internal / Confidential
Author	gitNoodler + Claude (Architecture)
Status	Active Development
Supersedes	v1 Iteration 1 (3-evaluator parallel grading)

This document defines the complete annotation rotation grading pipeline for WankrBot LLM training. It serves as the canonical reference for how responses are evaluated, annotated, cross-reviewed, and archived to produce high-quality training data.

TABLE OF CONTENTS

#	SECTION	DESCRIPTION
1	System Overview	Purpose, scope, and design principles
2	Annotator Pool	LLM evaluators and optional human grader
3	Pipeline Phases	End-to-end flow from trigger to archive
4	Rotation Mechanism	Latin square with randomization
5	Core Rule: No Self-Review	Anti-echo-chamber enforcement
6	Annotation Archive Schema	Data structure per archived layer
7	Yield Analysis	Annotations per response (math)
8	Grading Context	What each annotator receives
9	Stop Conditions	Saturation detection
10	Integration with Wankr	UI trigger and training ingestion
11	Iteration 1 Lessons	What v1 revealed and what v2 fixes
12	Cost and Rate Considerations	API budget management

1. SYSTEM OVERVIEW

Purpose

The Wankr Grading Pipeline is a manually-triggered, multi-LLM annotation system that produces high-density training data from curated WankrBot responses. It is not an automatic pipeline. The operator selects specific responses worth deep analysis and triggers the grading mechanism on demand.

Design Principles

- **Blind Independence** - Each annotator grades the original response without seeing others' work.
- **No Self-Review** - An annotator never sees its own prior annotation again after producing it.
- **Forced Cross-Pollination** - Each round exposes annotators only to foreign perspectives.
- **Randomized Rotation** - Review order is randomized per run to prevent positional bias.
- **Full Archival** - Every annotation from every round is copied and archived separately.
- **Manual Trigger** - The operator controls which responses enter the pipeline, managing cost and quality.

Scope

This system evaluates WankrBot's Grok-generated responses against the established persona composition (Theo Von, Chris D'Elia, J. Daniel Atlas, Jordan Belfort, Deep Crypto Degen), the focus integrity rules, hard/soft boundaries, and non-crypto guardrails defined in the v1 training documentation.

2. ANNOTATOR POOL

The system uses three primary LLM annotators plus an optional human evaluator. Each brings different evaluation tendencies, which is the core strength of the system.

ANNOTATOR	ROLE	API SOURCE	KNOWN BIAS (from v1)
ChatGPT	Evaluator #1	platform.openai.com	Moderate scorer (avg 81.2)
Grok	Evaluator #2 + Generator	console.x.ai	Self-lenient (avg 90.4)
Claude	Evaluator #3 (NEW in v2)	console.anthropic.com	TBD - first iteration
User	Human Evaluator (optional)	Manual input	Hardest grader (avg 53.0)

Note: Grok serves dual purpose as both the response generator and an evaluator. The no-self-review rule is especially critical for Grok since it evaluates its own outputs. In v1, Grok averaged 90.4 while User averaged 53.0 on the same responses - a 37-point gap that demonstrates why blind cross-review is essential.

3. PIPELINE PHASES

Phase 0: Trigger

The operator identifies a WankrBot response worth grading and triggers the pipeline via a "Send to Grading" action in the Wankr interface. The response (R) is captured along with its trigger post, intent category, and conversation context.

Phase 1: Blind Independent Annotation

All annotators receive Response R plus the grading context (persona rules, boundaries, penalty flags). Each produces an independent annotation without knowledge of others' work.

ANNOTATOR	INPUT	OUTPUT	THEN
ChatGPT	Response R + Context	Annotation A1	A1 locked away from ChatGPT forever
Grok	Response R + Context	Annotation A2	A2 locked away from Grok forever
Claude	Response R + Context	Annotation A3	A3 locked away from Claude forever
User	Response R + Context	Annotation A4	A4 locked away from User forever

CRITICAL: After Round 1, each annotator's own annotation is permanently removed from their context. They will never see it again in any subsequent round.

Phase 2: Rotation Review (Rounds 2-4)

Each annotator reviews the OTHER annotators' work in a randomized rotation. They produce new annotation layers reacting to foreign perspectives only.

Phase 3: Archive Collection

After all rounds complete (or saturation is reached), all archived annotation layers are collected into the training dataset with full metadata.

Phase 4: Training Ingestion

The annotated response with all layers feeds into WankrBot's training feedback loop, informing future Grok system prompt refinements and persona tuning.

4. ROTATION MECHANISM

The rotation follows a Latin Square pattern where each annotator reviews every other annotator's work exactly once across the rounds. The ORDER within each round is randomized per run to prevent positional bias.

4-Annotator Rotation (with User)

ANNOTATOR	ROUND 1 (on R)	ROUND 2	ROUND 3	ROUND 4
ChatGPT	Produces A1	Reviews A4	Reviews A3	Reviews A2
Grok	Produces A2	Reviews A1	Reviews A4	Reviews A3
Claude	Produces A3	Reviews A2	Reviews A1	Reviews A4
User (optional)	Produces A4	Reviews A3	Reviews A2	Reviews A1

The column order shown above is one example. Each run RANDOMIZES which annotation each reviewer sees first. The only constant: no annotator ever sees their own work.

3-Annotator Rotation (without User)

ANNOTATOR	ROUND 1 (on R)	ROUND 2	ROUND 3
ChatGPT	Produces A1	Reviews A3	Reviews A2
Grok	Produces A2	Reviews A1	Reviews A3
Claude	Produces A3	Reviews A2	Reviews A1

5. CORE RULE: NO SELF-REVIEW

This is the single most important rule in the pipeline. It eliminates confirmation bias and prevents echo chambers.

The Rule

After an annotator produces their annotation in any round, that annotation is **permanently removed** from their context for all future rounds. They will never see their own work again.

What Each Annotator Never Sees (4-annotator mode)

ANNOTATOR	NEVER SEES (across all rounds)
ChatGPT	A1, Round 2 output by ChatGPT, Round 3 output by ChatGPT, Round 4 output by ChatGPT
Grok	A2, Round 2 output by Grok, Round 3 output by Grok, Round 4 output by Grok
Claude	A3, Round 2 output by Claude, Round 3 output by Claude, Round 4 output by Claude
User	A4, Round 2 output by User, Round 3 output by User, Round 4 output by User

Why This Matters

- **Kills echo chambers** - Without this rule, an annotator would see their own prior critique and reinforce it rather than generating new insight.
- **Forces fresh perspective** - Each annotator must react only to foreign viewpoints, producing genuinely different annotation layers.
- **Reveals true consensus** - If two annotators independently flag the same issue without ever coordinating, that is high-confidence signal.
- **Exposes evaluator quality** - If one annotators reviews consistently generate more new layers from others, that annotator produces richer critiques.
- **Prevents Grok self-leniency** - v1 showed Grok scoring itself at 90.4 avg. This rule ensures Grok never reinforces its own generous self-evaluation.

6. ANNOTATION ARCHIVE SCHEMA

Every annotation produced in every round is immediately copied and archived as an independent record. This is critical: annotations are not overwritten or merged. Each one exists as a separate training data point.

Schema Per Archived Annotation

```
{
  "annotation_id":      "R-0042-claude-RND3-002",
  "response_id":        "R-0042",
  "original_response":  "...(full text of Grok response R)...",
  "author":             "claude",
  "round":              3,
  "reviewing":          "A1",
  "reviewing_author":   "chatgpt",
  "never_saw":          ["A3", "B3", "C3"],
  "rotation_seed":      "a7f3b2",
  "correction_type":    "logical_gap | tone | factual | persona_drift | ...",
  "severity":           "critical | moderate | minor",
  "content":            "...(the actual annotation text)...",
  "persona_flags": {
    "theo_von":         0.7,
    "delia":            0.3,
    "atlas":            0.8,
    "belfort":          0.9,
    "degen":            0.5
  },
  "penalty_triggers":  ["repetition", "off_topic"],
  "agreement_score":   2,
  "timestamp":         "2026-02-21T14:30:00Z"
}
```

Key Fields Explained

- **annotation_id** - Unique ID: ResponseID-Author-Round-Sequence
- **reviewing** - Which annotation this annotator was reviewing when they produced this layer
- **never_saw** - Complete list of their own prior outputs they were blocked from seeing
- **rotation_seed** - Random seed used for this runs rotation order (reproducibility)
- **persona_flags** - Per-influence scores (0-1) for how well the response hit each persona pillar
- **agreement_score** - How many OTHER annotators flagged a similar issue (0-3)

7. YIELD ANALYSIS

Each flagged response produces a predictable number of archived annotation layers.

The Math

MODE	ANNOTATORS	ROUNDS	TOTAL LAYERS	FORMULA
With User	4	4	16	4 annotators x 4 rounds
Without User	3	3	9	3 annotators x 3 rounds

Dataset Growth Projection

RESPONSES GRADED	LAYERS (4-ann)	LAYERS (3-ann)
10	160	90
50	800	450
100	1,600	900
500	8,000	4,500
1,000	16,000	9,000

Because the pipeline is manually triggered, dataset growth is controlled by the operator. Quality over quantity: 100 carefully selected responses with 1,600 layers is more valuable than 1,000 random responses.

8. GRADING CONTEXT

Every annotator receives identical context to ensure consistent evaluation criteria. This context is loaded into the system prompt for each API call.

Context Package (loaded for every annotator)

DOCUMENT	PURPOSE	SOURCE
Persona Influences	Score against the 5 persona pillars	wankr_v1_personaInfluences_generalized.pdf
Focus Integrity Rules	Did it stay on topic? Correct classification?	wankr_v1_focusIntegrityRules.pdf
Hard/Soft Boundaries	Did it violate any non-negotiable rules?	wankr_v1_standardhardsoftBounds.pdf
Non-Crypto Bounds	If off-topic, did it follow guardrails?	wankr_v1_noncryptoConversations.pdf
Persona Intro (10/10)	Reference for what perfect looks like	wankr_v1_Persona_Intro_10of10.txt
Penalty Flags (current)	Known issues to watch for	Updated per iteration

Annotation Prompt Structure

Each annotator receives a structured prompt that includes the grading context above, the response to evaluate, the specific annotation they are reviewing (Rounds 2+), and explicit instructions to produce corrections in a standardized format covering: factual accuracy, persona alignment per influence, tone and comedic value, focus integrity, boundary compliance, and overall effectiveness.

9. STOP CONDITIONS

Primary Stop: Round Completion

The pipeline runs for exactly N rounds where N = number of annotators. With 4 annotators, 4 rounds. With 3, 3 rounds. At that point every annotator has reviewed every other annotator's work exactly once.

Optional Early Stop: Saturation

If a round produces zero new unique correction types that weren't already captured in previous rounds, the pipeline can terminate early. This saves API costs when annotations have converged.

Saturation Detection

1. Compare `correction_type` + severity of new annotations against all prior archived layers.
2. If 100% of new annotations map to existing correction types with matching severity, saturation is reached.
3. The operator can override and force completion of all rounds regardless.

10. INTEGRATION WITH WANKR

UI Trigger

A "Send to Grading" button appears alongside each WankrBot response in the chat interface. Clicking it captures the response, its trigger context, and queues it for the grading pipeline. The operator can also batch-flag multiple responses and run grading as an overnight job.

API Keys Required

SECRET NAME	PROVIDER	STORED IN	PURPOSE
ANTHROPIC_API_KEY	Anthropic	Infisical	Claude annotations
OPENAI_API_KEY	OpenAI	Infisical	ChatGPT annotations
XAI_API_KEY	xAI	Infisical	Grok annotations + generation

Training Data Flow

- Operator flags response -> **grading queue**
- Pipeline runs rotation -> **16 (or 9) archived layers produced**
- Layers stored in -> **training/grading/**
- Training service ingests layers -> **persona tuning + penalty flag updates**
- Next iteration's penalty flags updated -> **grading context evolves**

11. ITERATION 1 LESSONS

Version 1 used parallel grading (3 evaluators scored independently, no cross-review). The following data informed the v2 design.

v1 Scoring Data (10 responses)

METRIC	USER	GROK	CHATGPT	AGGREGATE
Average Score	53.00	90.40	81.20	74.87
Highest Response	R2 (60)	R9 (96)	R9 (88)	R2 (79.00)
Lowest Response	R3-R10 (50)	R7 (83)	R7 (74)	R7 (69.00)
Score Range	50-60	83-96	74-88	69-79

Key Findings

- 37-point User-Grok gap - Grok was significantly more lenient grading its own outputs than the human operator.
- Most frequent penalty - Repetition patterns (Delia-style echoes, 'wait wait' intros, recycled catchphrases).
- Strongest persona signal - Belfort technical competence consistently emerged in on-chain analysis.
- Weakest area - Originality and comedic value scored lowest; formulaic structure penalized.
- Persona drift - Overuse of Delia-style repetition at the expense of other influences.

What v2 Fixes

- Adds Claude as 4th evaluator - new perspective not present in v1.
- Cross-review rotation - annotators react to each other's critiques, not just the raw response.
- No self-review - eliminates Grok's self-leniency bias entirely.
- Archived layers - 16 data points per response instead of 3 parallel scores.
- Randomized order - prevents systematic reviewer-position bias.

12. COST AND RATE CONSIDERATIONS

API Calls Per Graded Response

MODE	API CALLS	BREAKDOWN
4 annotators	16 total	4 (Round 1) + 4 (R2) + 4 (R3) + 4 (R4)
3 annotators	9 total	3 (Round 1) + 3 (R2) + 3 (R3)

Budget Recommendations

- Set **spending limits** on all three API accounts immediately (\$20-50/month during prototyping).
- Start with **3-annotator mode** (9 calls) to validate the pipeline before adding User as 4th.
- Batch grading runs during **off-peak hours** if rate limits are a concern.
- Monitor per-response cost and adjust batch size accordingly.
- The manual trigger design inherently controls cost - only grade what matters.

END OF DOCUMENT

WankrBot Grading Pipeline v2 - Annotation Rotation Mechanism

gitNoodler / Claude Architecture | February 2026